

SERIAL PERIPHERAL INTERFACE (SPI)

Complete RTL Design, UVM Verification & Technical Interview Handbook

Author: Aditya Raut	Target Environment: AMD Vivado 2025.1
Technology: Verilog / SystemVerilog	Verification Standard: UVM 1.2 (IEEE 1800.2)
Target Part: Zynq UltraScale+ (xczu3eg)	Pass Rate: 100% (0 Errors / 0 Mismatches)

1. Complete Protocol Concepts & Fundamentals

The Serial Peripheral Interface (SPI) is a synchronous, 4-wire, full-duplex serial bus developed by Motorola in the mid-1980s. Unlike I2C, SPI is a de facto standard without a single international governing committee. It was created as a 'Goldilocks' solution to connect microcontrollers to peripherals (EEPROMs, ADCs, sensors) without the high pin count of parallel buses or the baud-rate synchronization complexity of asynchronous UART.

1.1 Baseline 4-Wire Interface Signals

Signal	Driven By	Direction	Description & Hardware Function
SCLK	Master	Master -> Slave	Serial Clock. Timing reference generated by Master. Frequency configured via clock divider.
MOSI	Master	Master -> Slave	Master Out Slave In. Carries commands, addresses, and write data from Controller.
MISO	Slave	Slave -> Master	Master In Slave Out. Carries read data and status. Must be high-Z ('1'bz') when unselected!
CS_N	Master	Master -> Slave	Chip Select (active-low). Frames transaction window and activates target slave peripheral.

1.2 The 4 SPI Operating Modes (CPOL & CPHA)

SPI timing is governed by two parameters: Clock Polarity (**CPOL**) and Clock Phase (**CPHA**). A mode mismatch between Master and Slave is the #1 cause of bugs in SPI systems.

Mode	CPOL	CPHA	Clock Idle	Leading Edge	Trailing Edge	Sampling Occurs On
Mode 0	0	0	LOW (0V)	Rising (Sample)	Falling (Shift)	Rising Edge (First)
Mode 1	0	1	LOW (0V)	Rising (Shift)	Falling (Sample)	Falling Edge (Second)
Mode 2	1	0	HIGH (Vdd)	Falling (Sample)	Rising (Shift)	Falling Edge (First)
Mode 3	1	1	HIGH (Vdd)	Falling (Shift)	Rising (Sample)	Rising Edge (Second)

Crucial CPHA=0 Timing Detail: When CPHA=0, the very first clock edge is already a sample edge! Therefore, the MSB (bit 7) must be launched onto MOSI/MISO immediately when CS_N falls to 0, *before* SCLK begins toggling.

2. RTL Hardware Architecture & Code Breakdown

2.1 System Top-Level Wrapper (`spi\_top.sv`)

`spi\_top.sv` integrates the parameterized Master and Slave cores into a cohesive loopback and full-duplex verification setup. It implements a `tri1 spi\_miso\_bus` modeling the board-level pullup on MISO so that when CS_N is deasserted, the floating bus resolves to '1' instead of high-impedance 'z'.

2.2 Master Controller Architecture (`spi\_master.sv`)

The Master controller is structured as a robust 5-state synchronous FSM:

- **IDLE:** Holds CS_N=1, parks SCLK at configured CPOL, monitors start strobe.
- **CS_SETUP:** Asserts CS_N=0, waits CLK_DIV cycles for slave setup time (\$T_{CSS}\$). If CPHA=0, launches MSB onto MOSI.
- **TRANSFER:** Executes exactly \$2 \times \text{DATA_WIDTH}\$ clock half-cycles. Even/odd edges toggle SCLK and trigger shift/sample operations based on CPHA.
- **CS_HOLD:** Returns SCLK to CPOL idle level, holds CS_N=0 for CLK_DIV cycles (\$T_{CSH}\$ hold time).
- **COMPLETED:** Deasserts CS_N=1, pulses done=1 for 1 clock cycle, and returns to IDLE.

Master Core Sampling and Shifting Logic:

```
// spi_master.sv: Alternating Edge Detection Logic
if (edge_cnt[0] == 1'b1) begin // Trailing Edge
    if (cfg_cpha == 1'b0) begin
        mosi_reg <= tx_shift[DATA_WIDTH-2];
        tx_shift <= {tx_shift[DATA_WIDTH-2:0], 1'b0};
    end else begin
        rx_shift <= {rx_shift[DATA_WIDTH-2:0], spi_miso};
    end
end else begin // Leading Edge
    if (cfg_cpha == 1'b0) begin
        rx_shift <= {rx_shift[DATA_WIDTH-2:0], spi_miso};
    end else begin
        mosi_reg <= tx_shift[DATA_WIDTH-1];
        tx_shift <= {tx_shift[DATA_WIDTH-2:0], 1'b0};
    end
end
end
```

2.3 Reactive Slave Peripheral (`spi\_slave.sv`)

The Slave peripheral operates in the system clock domain using 2-stage flip-flop synchronizers on external SCLK, MOSI, and CS_N pins. This guarantees zero metastability and clean synthesis on Xilinx Zynq UltraScale+ FPGA architecture. Key features:

- **Tri-State MISO Output:** Driven as `assign spi\_miso = cs\_n\_active ? miso\_drive : 1'bz;`.
- **Preload Port:** `slave\_tx\_load` allows parallel preloading of return data prior to transmission.
- **Edge Alignment:** Evaluates `sclk\_rising` and `sclk\_falling` to sample and shift in lockstep with the active mode.

3. Modular UVM 1.2 Verification Suite

The verification environment is designed to enterprise UVM standards, providing a layered testbench with complete functional coverage and self-checking scoreboarding.

Component	UVM Base Class	Role & Implementation Details
spi_interface.sv	SystemVerilog interface	Bundles Master parallel, Slave parallel, and SPI serial bus. Features `cb_drv` and `cb_mon` clocking blocks for race-free synchronization.

spi_transaction.sv	<code>`uvm_sequence_item`</code>	Models transaction item with <code>`master_tx_data`</code> , <code>`slave_tx_data`</code> , <code>`cpol`</code> , <code>`cpha`</code> , <code>`delay`</code> , and captures <code>`master_rx_data`</code> , <code>`slave_rx_data`</code> .
spi_sequence.sv	<code>`uvm_sequence`</code>	Includes <code>`spi_corner_sequence`</code> (0x00, 0xFF, 0xAA, 0x55, walking bits), <code>`spi_modes_sequence`</code> (Modes 0-3), <code>`spi_random_sequence`</code> , and master regression.
spi_driver.sv	<code>`uvm_driver`</code>	Converts sequence items into pin transitions. Features a 6-cycle mode settling delay ensuring clock polarity stabilizes before CS_N assertion.
spi_monitor.sv	<code>`uvm_monitor`</code>	Passively observes bus activity, captures transmit and receive payloads upon <code>`master_done`</code> , and writes items to analysis ports.
spi_coverage.sv	<code>`uvm_subscriber`</code>	Covergroups tracking CPOL x CPHA cross coverage (100% of Modes 0, 1, 2, 3), Data ranges (min, max, alternating), and delays.
spi_scoreboard.sv	<code>`uvm_scoreboard`</code>	Dual-path comparator: validates MOSI path (<code>`slave_rx == master_tx`</code>) and MISO path (<code>`master_rx == slave_tx`</code>). Prints summary report table.
spi_agent.sv	<code>`uvm_agent`</code>	Encapsulates Sequencer, Driver, Monitor, and Coverage into a self-contained, portable block.
spi_env.sv	<code>`uvm_env`</code>	Top-level verification environment interconnecting agent monitor analysis port to scoreboard export.
spi_test.sv	<code>`uvm_test`</code>	Defines tests (<code>`spi_test`</code> , <code>`spi_rand_test`</code> , <code>`spi_modes_test`</code>) with objection drain time management.
spi_tb.sv	Testbench Module	Top simulation module: generates 100MHz clock, reset sequence, instantiates DUT, and triggers <code>`run_test()`</code> .

4. Vivado 2025.1 XSim Simulation Results

The complete suite was compiled, elaborated, and simulated using AMD Vivado 2025.1 native simulator tools (``xvlog``, ``xelab``, ``xsim``) with the Xilinx UVM 1.2 library. The scoreboard produced the following official report:

```

-----
                        SPI VERIFICATION REPORT
-----
Total SPI Transactions : 49
Mode 0 (0,0) Count    : 17
Mode 1 (0,1) Count    : 12
Mode 2 (1,0) Count    : 8
Mode 3 (1,1) Count    : 12
-----
MOSI Path Matches     : 49 / 49 (100.0%)
MOSI Path Mismatches  : 0
MISO Path Matches     : 49 / 49 (100.0%)
MISO Path Mismatches  : 0
-----
*** ALL SPI TRANSACTIONS MATCHED SUCCESSFULLY! ***
-----
UVM Report: INFO: 82 | WARNING: 0 | ERROR: 0 | FATAL: 0

```

5. Comprehensive SPI Interview Questions & Answers

This section covers high-yield technical interview questions asked for ASIC/FPGA Design and Verification roles.

Q1: Compare SPI, I2C, and UART across architecture, pins, speed, and protocol overhead.

• **Wires & Topology:** SPI uses 4 wires (push-pull) in point-to-point or dedicated-CS topology; I2C uses 2 open-drain wires with pullups in multi-master bus; UART uses 2 asynchronous wires point-to-point. • **Duplex:** SPI is natively full-duplex; I2C is half-duplex; UART is full-duplex. • **Speed:** SPI operates at 20-50+ MHz; I2C standard is 100kbps / 400kbps / 3.4Mbps; UART typical is 9600-115200 bps. • **Addressing & Overhead:** SPI uses hardware CS (0 address bits overhead); I2C requires 7-bit/10-bit address headers + ACK/NACK bits; UART requires start/stop framing bits.

Q2: Why must MISO be tri-stated when a Slave's CS_N is deasserted?

In multi-peripheral SPI architectures, all Slaves share the single MISO wire back to the Master. If an unselected Slave drives MISO low ('0') or high ('1') while the active Slave is transmitting, both drivers will fight, causing heavy bus contention, electrical damage, and corrupted data. Hence, unselected Slaves must drive high-impedance ('1'bz').

Q3: Explain the 'Linked Shift Register' concept in SPI.

The Master and Slave behave as a single circular ring buffer of size $2 \times \text{DATA_WIDTH}$. On every SCLK cycle, one bit shifts out of Master's MSB into Slave's LSB via MOSI, and one bit shifts out of Slave's MSB into Master's LSB via MISO. A 'Read' in SPI is never passive; the Master must clock the bus and transmit dummy bits on MOSI to pump read data back on MISO.

Q4: Why does Mode 0 (CPHA=0) require data to be valid before the first clock edge?

When CPHA=0, data is sampled on the first (leading) clock edge. For Mode 0, that edge is the rising edge. If the Slave or Master waited for the rising edge to launch data, the setup time requirement (T_{su}) would be violated. Therefore, bit 7 must be placed on the line immediately upon CS_N assertion.

Q5: What is clock polarity settling and why did you add it to the UVM Driver?

When switching between SPI modes (e.g. Mode 0 CPOL=0 to Mode 2 CPOL=1), SCLK transitions from 0 to 1 while the bus is idle. Because the Slave uses 2-stage flip-flop synchronizers to prevent metastability, the internal synchronizers require several clock cycles to register the new idle voltage level. Introducing a 6-cycle settling delay ensures the Slave's edge detector is fully stabilized before CS_N falls.

Q6: How does Clock Division work in an SPI Master?

The Master divides the system clock (F_{CLK}) using an integer counter `clk_cnt` running from 0 to `CLK_DIV - 1`. Each half-period of SCLK equals `CLK_DIV` cycles of `clk`. Therefore, the serial clock frequency is $F_{SCLK} = \frac{F_{CLK}}{2 \times \text{CLK_DIV}}$. For $F_{CLK} = 100 \text{ MHz}$ and $\text{CLK_DIV} = 4$, $F_{SCLK} = 12.5 \text{ MHz}$.

Q7: How did you verify SPI protocol compliance in UVM?

We implemented a layered UVM 1.2 environment with: 1. **Directed Sequences:** Sweeping all 4 modes (0, 1, 2, 3) and walking-bit corner vectors. 2. **Constrained-Random Sequences:** Random payloads with variable inter-packet delays. 3. **Dual-Path Scoreboard:** Independent verification of MOSI and MISO paths. 4. **Functional Coverage:** Full cross-coverage of CPOL x CPHA, data bins, and delays. Achieved 100% pass rate across 49 transactions in Vivado XSim.

Q8: What is the difference between standard SPI, Dual-SPI, and Quad-SPI (QSPI)?

• **Standard SPI:** 1 data line per direction (MOSI, MISO) -> 1 bit per SCLK cycle. • **Dual-SPI:** Converts MOSI and MISO into bidirectional half-duplex lines (IO0, IO1) -> 2 bits per SCLK cycle (doubles throughput for Flash reads). • **Quad-SPI (QSPI):** Uses 4 bidirectional data lines (IO0, IO1, IO2, IO3) -> 4 bits (nibble) per SCLK cycle. Widely used for FPGA boot ROMs and high-speed NOR flash.